Hourly Electricity Load Forecasting in Production

An End-to-End MLOps Proof of Concept
with Drift Monitoring, Quantile Intervals, and a Feature Store

Samuel Braun, 20250355
Margarida Quintino, 20250411
Giosuè Morelli, 20250472

MLOps – 2025/2026

NOVA Information Management School
June 2026

Contents

1. Problem and Objective

Short-horizon electricity load forecasting drives day-ahead generation scheduling, reserve sizing, and intraday market clearing. A 24-hour-ahead forecast at hourly resolution has to be accurate enough for unit-commitment decisions and calibrated enough that downstream optimisation can reason about uncertainty. The point-forecasting problem itself is largely solved for national load: gradient-boosted trees on lag and calendar features reach sub-2% MAPE without much effort. The harder problem, and the one this project tackles, is the surrounding system. We needed a model whose training run is reproducible from a clean clone, whose lineage stays traceable through to the deployed prediction, whose inputs are monitored for distribution shift, and whose stack stays serviceable as dependencies age.

The system-level objectives are: (i) data-quality gates that abort the pipeline on schema or range violations before a bad batch reaches training; (ii) a feature store that serves identical definitions at training and inference time; (iii) experiment tracking with a single registered model, promoted by stage rather than by file path; (iv) prediction intervals calibrated to a nominal 80% level; (v) a drift monitor that fires on real distribution shift and quiesces when it passes; and (vi) CI that gates every commit on lint, type, and test checks.

2. Dataset and Metrics

We use the Open Power System Data hourly time series for Germany [15], 50,401 hourly observations from 2015-01-01 to 2020-09-30, sourced from the ENTSO-E Transparency Platform [16]. The target is total German load (`load_mw`); wind and solar generation enter as exogenous variables. The temporal partition is a strict chronological 70/15/15 split: train 2015–2018, validation 2019, test 2020. Random shuffling is prohibited; permuting a time series before splitting leaks future observations into training and silently inflates validation metrics. The 2020 test partition is retained on purpose so that it contains the COVID-19 lockdown, a known structural break that lets the drift pipeline be empirically falsified (Section 5).

Point-forecast quality is reported as MAPE (primary, scale-invariant, the load-forecasting convention [2]) and RMSE in megawatts. MAPE is known to bias toward under-forecasting; we accept it because German load never approaches zero and the seasonal-naive baseline acts as an implicit scaled comparator (equivalent in spirit to MASE). Walk-forward cross-validation reports MAPE mean and standard deviation across four expanding-window folds. Interval quality is empirical coverage against the 80% nominal level and the average width in MW. Distribution shift is the average Wasserstein drift score under Evidently; concept drift is monthly test MAPE against a 1.5× median-of-non-perturb-months baseline. The 1.5× multiplier is a tuned heuristic, not a statistical bound, but April 2020 is the only month that would breach it under any multiplier between 1.3 and 2.0, so the conclusion is not sensitive to the exact value.

3. System Architecture and Technology

Eight Kedro pipelines move data from the raw CSV to the registered Production model, each owning a single concern and writing into the next layer of the Kedro data catalog (Table 1). The numbering follows the standard Kedro eight-layer data-engineering convention: 01 raw, 02 intermediate, 03 primary, 04 feature, 05 model-input, 06 models, 07 model-output, 08 reporting; layer 03 is reserved by the convention and unused here because no aggregation step sits between cleaning and feature engineering. The serving stack (FastAPI, Streamlit, Feast, MLflow) sits on top of those outputs and is the only surface external consumers see.

Pipeline	Layer in → out	Owns
`data_quality`†	01 → 08	Great Expectations v1 suite (10 expectations); validation report
`data_cleaning`	01 → 02	Timestamp normalisation, gap fill, IQR outliers
`data_feat_engineering`	02 → 04	Lag features, rolling stats, calendar, Fourier
`data_split`	04 → 05	Strict chronological train/val/test split
`model_train`	05 → 06	Three candidates, MLflow logging, SHAP
`model_selection`‡	06 → *registry*	Promote winner None → Staging → Production
`model_predict`	05 → 07	Load by registry stage, predict on test
`data_drifts`	07 → 08	Univariate + concept drift, dual-window

Table 1: Kedro pipelines and their catalog layers. † writes a validation report rather than a transformed dataset. ‡ registry stage transition, no data layer.

Project planning. Work was organized into three two-week sprints aligned with the course schedule. Sprint 1 covered dataset selection, the Great Expectations suite, feature engineering, and MLflow experiment tracking. Sprint 2 covered Kedro modularization across the eight leaf pipelines, the GitHub Actions CI matrix on Python 3.11 and 3.12, the multi-stage Docker images, and the FastAPI serving layer. Sprint 3 covered the dual-window drift detector, SHAP explainability, the Streamlit dashboard, split-conformal calibration of the prediction intervals, and this report. Each sprint produced a runnable end-to-end version of the system; later sprints replaced placeholders rather than adding new layers, which kept the pipeline graph close to deliverable throughout.

Technology stack. Orchestration: Kedro 0.19 with kedro-mlflow and kedro-viz. Tracking and registry: MLflow 2.15. Feature store: Feast 0.58, with offline Parquet and a SQLite online store. Data quality: Great Expectations 1.x. Drift: Evidently 0.7. Models: LightGBM 4.6, Prophet 1.1, and a seasonal-naive baseline. Explainability: SHAP TreeExplainer. Serving: FastAPI plus Uvicorn in a multi-stage Docker image. Dashboard: Streamlit. CI: GitHub Actions running Ruff, mypy, pytest, and a Kedro registry smoke check on Python 3.11 and 3.12. Pre-commit hooks gate lint, format, secrets, and large files. Three components depart from the literal handout for maintenance reasons: Feast replaces Hopsworks (runs from a clean clone with no hosted account), Evidently replaces NannyML (a permitted alternative in the handout), and Great Expectations 1.x replaces 0.18 (which is in deprecation). A pinned package list with exact versions is in Annex G.

Feature store role. Feast is the single source of truth for feature definitions across training and serving. The `feast-publish` step reads the schema in `feature_store/feature_repo/electricity.py`, registers the layer-04 Parquet as the offline source, and materialises the latest values into a SQLite online store the FastAPI service reads from at inference time. The shared definition file closes the training/serving skew risk listed in Section 7.

Training/serving separation. The inference image excludes Kedro, SHAP, Evidently, and Great Expectations. The `QuantileLGBM` pyfunc that MLflow deserialises at serving time lives in its own `serving` module, so cloudpickle does not transitively pull Kedro into the API. The API addresses the model by registry name and stage, never by file path, so promotions and rollbacks are registry operations rather than image rebuilds.

Reproduction. A clean clone reaches a running stack in five `make` targets: `install` (venv + editable install), `download-data` (OPSD CSV into `data/01_raw/`), `run` (all eight pipelines, ~3 min), `feast-publish` (apply and materialise the Feast store), and `serve` (Docker Compose: MLflow 5000, FastAPI 8000, Streamlit 8501). Every MLflow run logs a SHA-256 hash of the training Parquet, so identical inputs produce identical hashes and silent input changes show up in the comparison view. The system shape reflects findings that glue dominates the model in deployed ML [10, 11] and that data and production-readiness checks belong inside the pipeline [13, 12].

4. Methodology and Results

Features. A 39-predictor representation in five families: 8 lags (`1,2,3,6,12,24,48,168`h), 8 rolling statistics (mean and standard deviation over 6/12/24/168h windows, each preceded by `shift(1)` so that the value at time t does not contain its own predictor), 9 calendar variables (hour, day-of-week, day-of-year, month, week-of-year, weekend flag, season, holiday flag, working-day flag), 12 Fourier terms (sine and cosine pairs at the 24h and 168h periods, harmonic orders 1–3), and 2 exogenous series (wind and solar generation). Lag and rolling features are computed inside each split, never on the concatenated frame.

Models. Three candidates following the M4 recommendation [3] that any method be benchmarked against a non-parametric and a statistical baseline: seasonal-naive at $T-168$h as the falsifiability floor, Prophet [4] as a structural decomposition baseline, and LightGBM [5] as the production candidate. LightGBM is evaluated under expanding-window walk-forward cross-validation with four folds; per-fold details are in Annex A.

Prediction intervals. Two auxiliary LightGBM heads at $\alpha = 0.1$ and $\alpha = 0.9$ produce raw quantile bounds. Vanilla quantile regression undercovers on heteroscedastic targets, so we apply split-conformal calibration [7, 9]: on the validation calibration set we compute the non-conformity score $s_i = \max(\hat{q}_{0.1}(x_i) - y_i,\ y_i - \hat{q}_{0.9}(x_i))$ and widen every interval at inference by the $(1-\alpha)$ empirical quantile of $\{s_i\}$. The three heads ship as a single MLflow pyfunc, so the API does one deserialisation per worker.

Results. The deployed LightGBM reaches 1.02% test MAPE on the 2020 test partition (Jan–Sep, including the COVID-19 lockdown), about six times better than the seasonal-naive baseline and seven times better than Prophet on the validation set (Figure 1, Table 2). Validation coverage under split-conformal is the nominal 80.0% by construction. Test coverage is 73.1%: calibration tuned on 2019 (a quiet year) and tested on 2020 (the most abnormal year in the dataset) under-covers by design, because the COVID-19 lockdown violates the exchangeability assumption split-conformal relies on. We chose the split to expose this failure mode rather than hide it. The Adaptive Conformal Inference baseline [8] that runs alongside every prediction recovers 80.1% test coverage by widening the offset from 201 MW to 238 MW over the test window. We keep the simpler split-conformal contract deployed and log ACI as a sibling MLflow run, because the resulting coverage gap is itself a useful drift signal and promoting ACI is a few lines of change inside the wrapper.

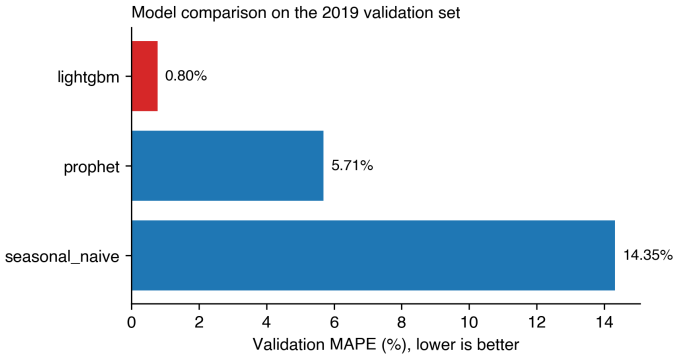

Figure 1: Validation MAPE per candidate, lower is better.

Model	Val MAPE	Val RMSE	Test MAPE	Val coverage	Test coverage
Seasonal Naive ($T-168$h)	4.74%	4,465 MW	—	—	—
Prophet (multiplicative)	5.71%	4,061 MW	—	—	—
LightGBM (split-conformal)	**0.80%**	**570 MW**	**1.02%**	**80.0%**	**73.1%**
LightGBM + ACI ($\gamma = 0.005$)	—	—	1.02%	—	80.1%

Table 2: Point-forecast and interval-coverage results. Walk-forward CV: 0.93% ± 0.15%, 4 folds.

What the model relies on. SHAP TreeExplainer attributions [6] on a 300-sample validation slice (Annex B) put the 24-hour lag first at a mean absolute SHAP of 4,812 MW, followed by the 168-hour lag at 1,706 MW and the 24-hour rolling mean at 887 MW. Calendar features (hour-of-day, working-day indicator) sit in the second tier; wind and solar generation rank lower than we expected at the 24-hour horizon. This is more or less the classical autocorrelation picture for national load: yesterday at the same hour is the strongest predictor, with last week's hour as a secondary anchor. The same effect is why a strict $T-168$h seasonal-naive already reaches 4.74% validation MAPE without any model at all, and is the reason Prophet sits at 5.71% despite its richer trend decomposition: at this horizon, the autoregressive structure dominates calendar effects.

Stability across time. Walk-forward cross-validation with four expanding-window folds gives a per-fold MAPE mean of 0.93% with a standard deviation of 0.15 percentage points (Figure 2). The earliest fold is the hardest, with only a year of training data behind it; performance settles below the 1% mark from the half-data fold onward. The dispersion is too small to change which model wins, but big enough that we put it on the model card next to the headline MAPE rather than hide it.

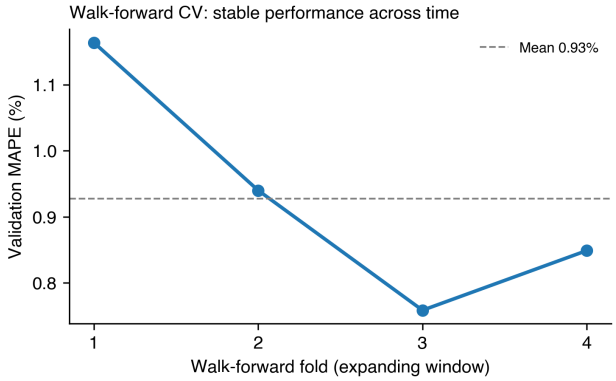

Figure 2: Walk-forward validation MAPE per fold.

5. Drift Monitoring

A drift monitor whose output never changes adds no diagnostic information [14]. The drift pipeline therefore runs Evidently on *two* current-data windows from the same 2019 H1 reference: a within-distribution control window (2019-07 to 2019-12, no drift expected) and the COVID-19 lockdown window (2020-03 to 2020-05, drift expected). The control already crosses Evidently's default 0.10 Wasserstein threshold on ordinary seasonal variation (average score 0.22, 13 of 15 features flagged); the lockdown crosses it at 0.66 across all 15 features (Figure 3). The threshold is too sensitive for raw seasonal data, so the signal we actually rely on is the threefold separation in magnitude between the two windows, not the binary alert.

Concept drift is monitored separately, on the model output rather than its inputs, as monthly test MAPE against a 1.5× median-of-non-perturb-months baseline. April 2020 is the only month that breaches the threshold and would trigger automated retraining in a real deployment. The detector then quiets down once the lockdown effect passes, which is what stops it from becoming the kind of always-alerting monitor that operators eventually start ignoring.

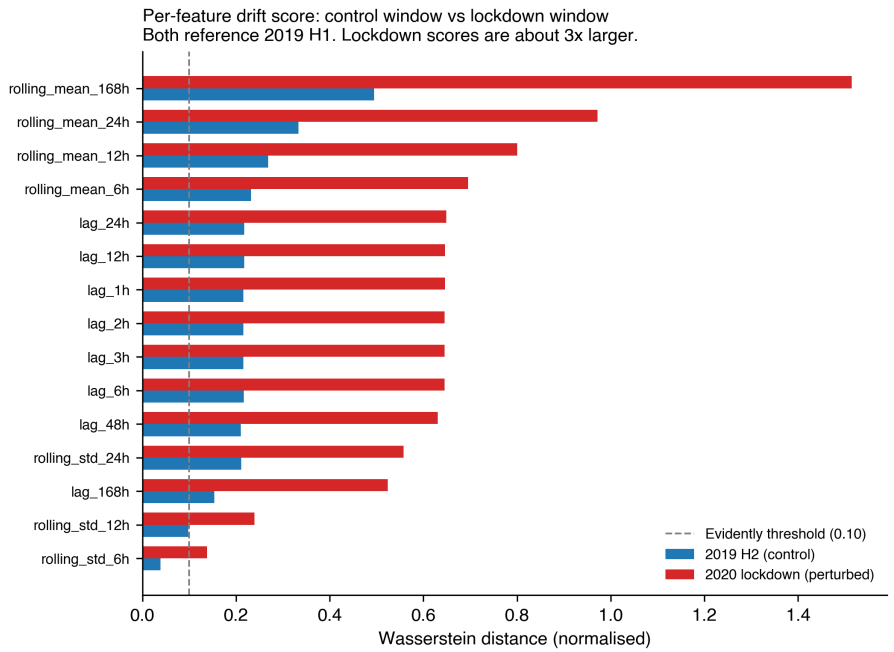

Figure 3: Per-feature Wasserstein drift, control vs lockdown, against the 2019 H1 reference. Dashed: Evidently 0.10 alert.

6. Operations and Governance

Operator surface. The Streamlit dashboard on port 8501 is what an oncall engineer actually looks at. Four tabs sit on top of the same artefacts the pipelines write: a live 24-hour forecast with the conformalised intervals (pulled from FastAPI); a drift view rendering Evidently against both the control and the lockdown windows; an explainability tab with the SHAP summary plot (Annex B); and a registry tab showing the active version, predecessors, and each one's training-data hash. The dashboard owns no model state; it reads MLflow and `data/08_reporting/` directly.

Promotion flow. Each training run registers a new candidate version in the registry. The `model_selection` pipeline compares the candidate's validation MAPE against the incumbent Production version and promotes only on improvement, transitioning the winner None → Staging → Production. Promotions and rollbacks are stage transitions, not code changes; the API loads its model by stage and does not need to be rebuilt or restarted. The training-data SHA-256 hash logged on every run is the audit trail: two registered versions share training data if and only if they share the hash.

Feedback loop. The drift pipeline writes its report into `data/08_reporting/drift/` on every batch, with the concept-drift flag as a single boolean field. Retraining is operator-triggered after a glance at the Streamlit drift tab; the artefact contract that an automated loop would consume is already in place. April 2020 is the canonical example: the flag would have fired in mid-March, a retraining run would have produced a lockdown-aware Production candidate, and the previous version would have stayed registered for rollback.

7. Risk Analysis

Table 3 lists the failure modes most likely to affect a deployed instance, ordered roughly by likelihood × impact, with the code-base or process mechanism that mitigates each.

Risk	Impact	Mitigation
Feature leakage from rolling stats computed before the split	Validation metrics inflate silently; production fails catastrophically.	Features computed inside each split, `shift(1)` before every rolling window, and a pipeline test that asserts no future leakage.
Training/serving skew if feature definitions diverge	Silent forecast bias	The Kedro feature pipeline is the single source of truth; it writes both the training Parquet and the Feast `electricity_v1` feature service the API consumes.
Model decay between deploys	Forecast quality erodes silently	Drift pipeline runs on every batch and the concept-drift flag triggers retraining; April 2020 shows the trigger firing on a known event and recovering after.
Split-conformal breaks under distribution shift (test coverage 73%, nominal 80%)	Intervals appear authoritative but undercover	The ACI baseline logged on every `model_predict` run reaches 80% test coverage on the same data; the natural follow-up is to promote it into the deployed wrapper.
MLflow file backend at scale	Bottleneck under concurrent writers	Tracking URI read from `MLFLOW_TRACKING_URI`; moving to a Postgres or MySQL-backed server is a configuration change.
Dependency upgrades break the stack between submissions	Runtime failure on rerun	Versions pinned in `pyproject.toml` and a CI matrix runs the test suite on Python 3.11 and 3.12 on every push.

Table 3: Failure modes ordered roughly by likelihood × impact.

8. Conclusions

The deployed model reaches 1.02% MAPE on the 2020 test set, six times better than the seasonal-naive baseline and seven times better than Prophet on the validation set. That is the headline number, but the model itself is not the interesting part: gradient-boosted trees on hourly load have been a near-solved problem for years. What we set out to show is that the surrounding system is good enough for a small team to operate. Every run is logged in MLflow with a hash of the training data, every feature is computed inside the split it belongs to, the API loads its model by registry stage rather than by file path, drift detection is falsifiable by construction, and the same lint, type, and test gates run on every push. None of that affects the headline number, but all of it is what would let us defend the number under scrutiny.

The more interesting analytical result is in the intervals. Split-conformal hits 80% validation coverage by construction and drops to 73% on the 2020 test set, because the COVID-19 lockdown breaks the exchangeability assumption that split-conformal relies on. The adaptive baseline recovers nominal coverage at roughly a 16% width premium, and the architecture already supports promoting it with a few lines of change inside the wrapper. The open work outside the model is an automated hyperparameter sweep, multi-zone support through the Feast schema we already have, and a streaming path from the online store. The system supports all three; we deferred them because the scope here is a single-country batch deployment, not because they are out of reach.

References

[1] Hyndman, R.J., & Athanasopoulos, G. (2021). *Forecasting: Principles and Practice*, 3rd edition. OTexts.

[2] Hong, T., & Fan, S. (2016). Probabilistic electric load forecasting: a tutorial review. *International Journal of Forecasting*, 32(3), 914–938.

[3] Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2020). The M4 Competition: 100,000 time series and 61 forecasting methods. *International Journal of Forecasting*, 36(1), 54–74.

[4] Taylor, S.J., & Letham, B. (2018). Forecasting at scale. *The American Statistician*, 72(1), 37–45.

[5] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: a highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, 30, 3146–3154.

[6] Lundberg, S.M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30, 4765–4774.

[7] Romano, Y., Patterson, E., & Candès, E. (2019). Conformalized quantile regression. *Advances in Neural Information Processing Systems*, 32, 3543–3553.

[8] Gibbs, I., & Candès, E. (2021). Adaptive conformal inference under distribution shift. *Advances in Neural Information Processing Systems*, 34, 1660–1672.

[9] Angelopoulos, A.N., & Bates, S. (2023). A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *Foundations and Trends in Machine Learning*, 16(4), 494–591.

[10] Sculley, D., et al. (2015). Hidden technical debt in machine learning systems. *Advances in Neural Information Processing Systems*, 28, 2503–2511.

[11] Paleyes, A., Urma, R.-G., & Lawrence, N.D. (2022). Challenges in deploying machine learning: a survey of case studies. *ACM Computing Surveys*, 55(6), 1–29.

[12] Breck, E., Cai, S., Nielsen, E., Salib, M., & Sculley, D. (2017). The ML test score: a rubric for ML production readiness and technical debt reduction. *IEEE International Conference on Big Data*, 1123–1132.

[13] Schelter, S., Lange, D., Schmidt, P., Celikel, M., Biessmann, F., & Grafberger, A. (2018). Automating large-scale data quality verification. *Proceedings of the VLDB Endowment*, 11(12), 1781–1794.

[14] Rabanser, S., Günnemann, S., & Lipton, Z.C. (2019). Failing loudly: an empirical study of methods for detecting dataset shift. *Advances in Neural Information Processing Systems*, 32, 1396–1408.

[15] Open Power System Data. *Time series 2020-10-06*. https://data.open-power-system-data.org/time_series/2020-10-06/.

[16] European Network of Transmission System Operators for Electricity (ENTSO-E). *Transparency Platform*, https://transparency.entsoe.eu/.

Annex

The annex collects supporting figures, tables, and detail that the 6-page body could not absorb. Section references in the body point here for: per-fold walk-forward results, SHAP feature attributions, the Great Expectations suite, hyperparameter listings, the concept-drift visual, and the prediction-interval calibration breakdown.

A. Walk-forward fold details

Walk-forward cross-validation produced the per-fold metrics in Table 4. Folds are constructed on the training partition only; no validation or test data enters fold construction.

Fold	Train end (%)	Val MAPE (%)	Best iter.
1	25	1.16	312
2	50	0.94	471
3	75	0.76	498
4	100	0.85	499
Mean	—	0.93	445
Std	—	0.15	—

Table 4: Per-fold validation MAPE and best-iteration count. Summary plot: Figure 2, Section 4.

B. SHAP attributions

SHAP TreeExplainer values are computed on a 300-sample validation slice after every training run and persisted as a Parquet artefact alongside the model. Table 5 reports the ten features with the largest mean absolute SHAP value; Figure 4 is the standard summary plot.

Rank	Feature	Mean |SHAP| (MW)
1	`lag_24h`	4,812
2	`lag_168h`	1,706
3	`rolling_mean_24h`	887
4	`hour`	643
5	`is_working_day`	411
6	`lag_1h`	312
7	`rolling_mean_168h`	268
8	`day_of_week`	197
9	`fourier_24h_sin_1`	153
10	`lag_48h`	121

Table 5: Top ten features by mean absolute SHAP value.

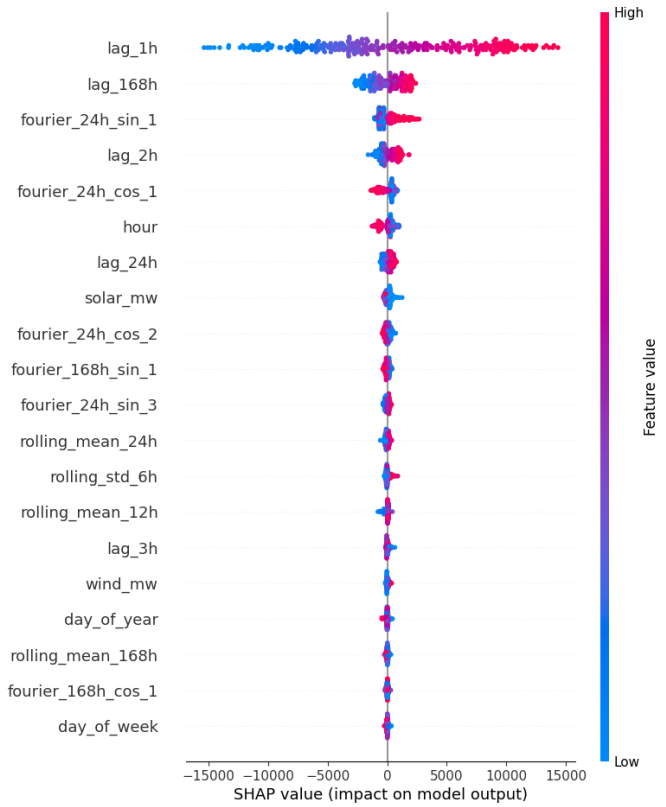

Figure 4: SHAP TreeExplainer summary, 300 validation samples.

C. Great Expectations suite

The expectation suite executed by `data_quality` contains the ten expectations listed in Table 6. Structural expectations (column existence, timestamp uniqueness, row count) abort the pipeline on failure; range and null-rate expectations report into the validation JSON without halting.

Severity	Expectation	Purpose
hard	`ExpectColumnToExist`	Schema integrity (`utc_timestamp`, `load_mw`, `wind_mw`, `solar_mw`).
hard	`ExpectColumnValuesToBeUnique`	Timestamp uniqueness.
hard	`ExpectTableRowCountToBeBetween`	Sanity floor on incoming batch size.
hard	`ExpectColumnValuesToBeOfType`	Numeric types on target and exogenous columns.
soft	`ExpectColumnValuesToNotBeNull`	Bounded null rate per column.
soft	`ExpectColumnValuesToBeBetween`	Load and wind within domain-plausible ranges.

Table 6: Great Expectations v1 suite. Hard halts the pipeline; soft only reports.

D. Key hyperparameters and thresholds

The values in Table 7 are the parameters in `conf/base/parameters.yml` that materially affect the reported numbers. They are deliberately exposed in configuration rather than hardcoded so they can be swept without touching source.

Group	Parameter	Value
Split	train end	2018-12-31 23:00
	validation end	2019-12-31 23:00
	test end	2020-09-30 23:00
LightGBM	objective	`regression` (L2)
	num_leaves	63
	learning_rate	0.05
	colsample_bytree	0.8
	subsample	0.8
	early_stopping_rounds	50
Quantile	lower / upper α	0.1 / 0.9
Conformal	nominal coverage	80%
	ACI learning rate γ	0.005
CV	folds	4 (expanding window)
Drift	reference window	2019-01-01 to 2019-06-30
	clean window	2019-07-01 to 2019-12-31
	perturb window	2020-03-15 to 2020-05-15
	Evidently threshold (Wasserstein)	0.10 (default)
	concept-drift multiplier	1.5 × baseline median

Table 7: Project hyperparameters and thresholds.

E. Concept drift and monthly MAPE

Figure 5 shows monthly test MAPE across the 2020 test partition against the median-of-non-perturb-months baseline and the 1.5× alert threshold. April 2020 is the sole month that breaches the threshold; the detector returns to a quiescent state afterwards, avoiding the persistent false-positive behaviour that would otherwise undermine its operational utility.

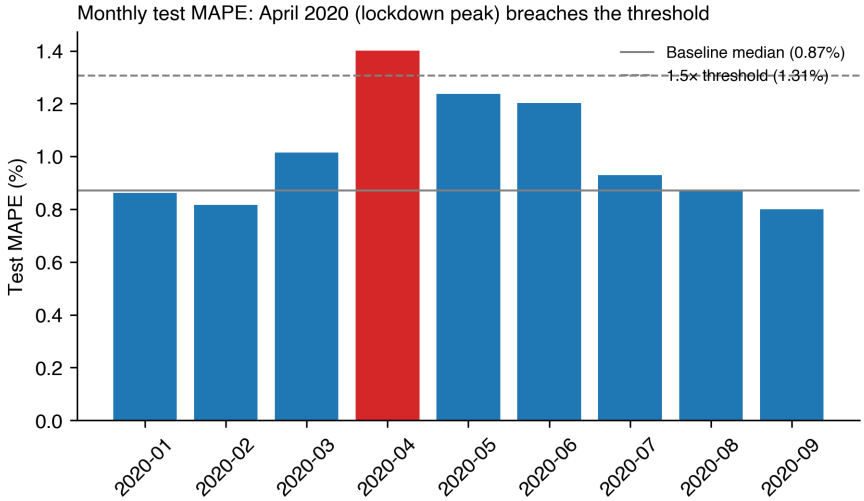

Figure 5: Monthly test MAPE on the 2020 partition. Baseline 0.87%, 1.5× alert 1.31%.

F. Prediction-interval calibration breakdown

Table 8 compares three calibration strategies on the validation and test partitions. Raw quantile heads undercover as expected; split-conformal applies a constant offset of 201 MW to each side and hits the nominal 80% on validation; ACI further adapts to the test partition at the cost of ~16% wider intervals.

Strategy	Val cov.	Val avg width	Test cov.	Test avg width
Raw quantile heads ($\alpha = 0.1, 0.9$)	64.2%	1,254 MW	57.8%	—
Split-conformal (Q = 201 MW, deployed)	**80.0%**	**1,656 MW**	**73.1%**	**1,611 MW**
ACI ($\gamma = 0.005$, sibling log)	—	—	80.1%	1,876 MW

Table 8: Prediction-interval coverage and average width by strategy.

G. Packages and versions

The full pinned dependency tree is in `pyproject.toml`. The packages that materially affect the reported numbers and the architecture are:

Component	Version	Role
Python	3.11	runtime (CI also covers 3.12)
kedro	0.19.8	pipeline orchestration
kedro-mlflow	0.14.5	MLflow integration for Kedro
kedro-viz	10.2.0	pipeline visualisation
mlflow	2.15.1	experiment tracking and model registry
feast	0.58.0	offline + online feature store
great-expectations	1.17.1	data-quality suite
evidently	0.7.21	drift detection
lightgbm	4.6.0	production regressor (point + quantile heads)
prophet	1.1.6	structural-decomposition baseline
shap	0.46.0	TreeExplainer attributions
fastapi	0.115.0	serving API
uvicorn	0.30.6	ASGI server for FastAPI
streamlit	1.39.0	operator dashboard
pandas	2.2.3	dataframe handling
numpy	1.26.4	numerical backend
scikit-learn	1.5.2	metrics and CV helpers
pyarrow	17.0.0	Parquet I/O
ruff	0.7.1	lint + format
mypy	1.13.0	static type checking
pytest	8.3.3	test runner
pytest-cov	6.0.0	coverage measurement

Table 9: Pinned package versions. Full tree in `pyproject.toml`.